Chapter 5
Public-Key Cryptography, Integrity and Trust

RSA from the arithmetic up, hashes and what they cannot do, MACs versus signatures, and the chain of certificates that ends in your operating system

Information Security

Based on the asymmetric cryptography, hashing, PKI and steganography lecture and the OpenSSL TP

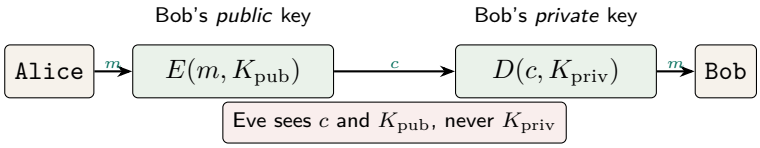
How to read this chapter

Chapter 4 ended with a shared secret agreed over a public channel and no way to know who was on the other end. This chapter supplies the missing half. It starts with why a key *pair* changes the distribution problem (Section 1), builds the four pieces of arithmetic RSA needs (Section 2), then RSA itself with the slide and TP numbers worked by hand and the three practical caveats that turn textbook RSA into usable RSA (Section 3). Integrity needs a different primitive: hash functions, their three properties, the birthday bound and the password-storage story (Section 4). A hash alone cannot authenticate, so Section 5 adds a secret in the two possible ways, MAC and signature, and shows what each can and cannot promise. Signatures need authentic public keys, which is the recursion that certificates and PKI resolve (Section 6). Section 7 is the lecture's coda on steganography. Section 8 maps the OpenSSL TP's RSA missions onto the chapter and Section 9 collects traps and classic problems.

1 Why a Key Pair

1.1 The counting argument and the mailbox

With one shared key per pair of users, N people need $\binom{N}{2} = N(N-1)/2$ keys and a secure channel for each: 6 people, 15 keys; 100 people, 4,950. With key pairs each person publishes one key and keeps one: $2N$ keys, 12 and 200 respectively, and no pre-shared secret. The model is a mailbox: anyone can drop a letter through the slot, only the owner has the key to open the box.



1.2 One-way and trapdoor functions

The mathematical object behind the mailbox is a **trapdoor one-way function**. A one-way function $y = f(x)$ is easy to compute and infeasible to invert. A trapdoor function is one-way for everyone except the holder of a secret t, generated together with f: the public key is f, the private key is t. The candidate the course uses is integer factorisation: 6,895,601 is a product of two primes and finding them takes work, but given the trapdoor $t = 1931$ the division $6{,}895{,}601/1931 = 3571$ is immediate. A public-key system must make key generation, encryption with the public key and decryption with the private key *easy*, and deriving the private key from the public one, or the message from the ciphertext, *infeasible*.

This is the price. Alice no longer needs a secret delivered in advance, but she must know that the public key she holds is really Bob's; if Eve can substitute her own, Eve reads everything. Sections 5 and 6 are about paying that price.

Key insight. Public keys remove the need for a secret channel and create the need for an authentic one.

Hard questions, answered

Q. Why does asymmetric cryptography not eliminate the key-distribution problem but transform it?
A. Secrecy of the distributed key is no longer needed; authenticity is. A wrong public key gives Eve the plaintext.

Q. For $N = 100$ users, how many keys under each model?
A. Symmetric $100 \cdot 99/2 = 4{,}950$; asymmetric 200.

2 The Arithmetic RSA Needs

Four facts, each used exactly once in Section 3.

Congruences. $a \equiv b \pmod{n}$ means n divides $a - b$; computations "mod n" keep only remainders in $0..n-1$, and you may reduce after every multiplication.

Inverses. a has a multiplicative inverse modulo n, a number a^{-1} with $a \cdot a^{-1} \equiv 1$, exactly when $\gcd(a, n) = 1$. For tiny numbers try $d = 1, 2, \ldots$ until $ad \bmod n = 1$; in general the extended Euclidean algorithm finds it in $O(\log n)$ steps.

Euler's totient. $\varphi(n)$ counts the integers in $1..n-1$ that are coprime to n. For a prime, $\varphi(p) = p - 1$. For a product of two distinct primes, $\varphi(pq) = (p-1)(q-1)$: of the $pq - 1$ candidates, $q - 1$ are multiples of p and $p - 1$ are multiples of q, and $pq - 1 - (q-1) - (p-1) = (p-1)(q-1)$.

Euler's theorem. If $\gcd(a, n) = 1$ then $a^{\varphi(n)} \equiv 1 \pmod{n}$. Why: multiplying by a permutes the $\varphi(n)$ residues coprime to n (it is injective because a is invertible), so the product of all of them equals the product of all of them times $a^{\varphi(n)}$, and cancelling the invertible product leaves $a^{\varphi(n)} \equiv 1$. For a prime this is Fermat's little theorem, $a^{p-1} \equiv 1 \pmod{p}$.

Technique: Fast exponentiation, the only way to compute $c^d \bmod n$ by hand

Square repeatedly, reduce modulo n at every step, and multiply together the powers whose exponents sum to d. $26^7 \bmod 33$: $26^2 = 676 \equiv 16$; $26^4 \equiv 16^2 = 256 \equiv 25$; and $7 = 4 + 2 + 1$, so $26^7 \equiv 25 \cdot 16 \cdot 26 = 10{,}400 \equiv 5$. Cost $O(\log d)$ multiplications instead of d. Never compute 26^7 as an integer and reduce at the end; the numbers explode and the exam does not give you a calculator that holds them.

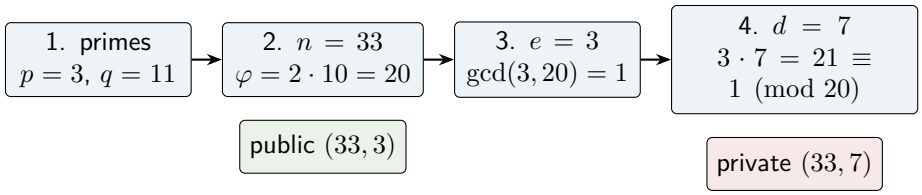
3 RSA

3.1 The algorithm

History. Rivest, Shamir and Adleman, 1977; the same scheme was found at GCHQ by Cocks in 1973 and kept secret.

1. Choose two distinct large primes p, q, random and of similar size.
2. $n = pq$ and $\varphi = (p-1)(q-1)$.
3. Choose e with $1 < e < \varphi$ and $\gcd(e, \varphi) = 1$.
4. Compute d with $ed \equiv 1 \pmod{\varphi}$.

Public key (n, e); private key (n, d); p, q and φ stay secret (or are destroyed). Encrypt $c = m^e \bmod n$; decrypt $m = c^d \bmod n$, for messages $0 \le m < n$.



3.2 Why decryption undoes encryption

$ed = 1 + k\varphi$ for some integer k. If $\gcd(m, n) = 1$, Euler gives $m^{ed} = m \cdot (m^{\varphi})^k \equiv m \cdot 1^k = m \pmod{n}$. If p divides m, both sides are 0 modulo p, and modulo q (which does not divide m) Fermat gives $m^{ed} = m \cdot (m^{q-1})^{k(p-1)} \equiv m$; a congruence that holds modulo p and modulo q holds modulo pq. The case $q \mid m$ is symmetric. So $(m^e)^d \equiv m$ for every m, and the two exponents can be used in either order, which is what signatures will exploit.

Worked example: The slide numbers and the TP password

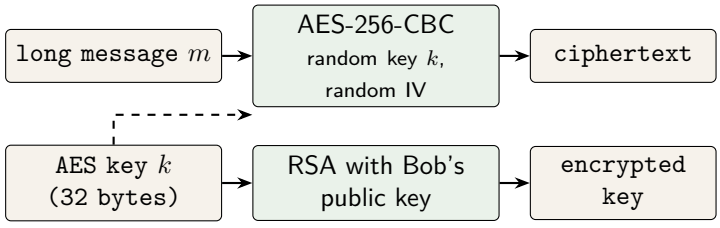
Slide, $n = 33$, $e = 3$, $d = 7$. $m = 4$: $c = 4^3 = 64 \bmod 33 = 31$; $31^7 \bmod 33 = 4$. Exercise $m = 5$: $c = 125 \bmod 33 = 26$; $26^7 \bmod 33 = 5$ by the fast-exponentiation example above.
TP Mission 1: $p = 5$, $q = 11$, $e = 3$, $C = 23$. $n = 55$, $\varphi = 40$; d: $3d \bmod 40 = 1$ gives $d = 27$ ($81 = 2 \cdot 40 + 1$). Decrypt: $23^2 = 529 \equiv 34$; $23^4 \equiv 34^2 = 1156 \equiv 1$; so $23^{27} = (23^4)^6 \cdot 23^3 \equiv 23^3 = 23 \cdot 34 = 782 \equiv 12$. The password is $m = 12$. Verify: $12^3 = 1728 = 31 \cdot 55 + 23 = C$. (Observation the TP asks for: re-encrypting gives back exactly C, because textbook RSA is deterministic.)
A fresh pair: $p = 7$, $q = 13$, $e = 5$: $n = 91$, $\varphi = 72$, $d = 29$ ($145 = 2 \cdot 72 + 1$); $m = 2 \mapsto c = 32$.

3.3 Security: factoring, size and speed

Whoever factors n computes φ and then d, so RSA is at most as strong as factoring. The TP's 50-bit modulus falls to trial division up to $\sqrt{n} \approx 2^{25} \approx 3.4 \times 10^7$ in seconds; a 512-bit modulus has $\sqrt{n} \approx 2^{256}$ and needs the number field sieve (512 bits is deprecated, but not because of trial division). Three facts turn the textbook scheme into a usable one.

In the real world. RSA-768 (232 digits) was factored in December 2009, RSA-250 (829 bits) in February 2020. 2048 bits is the default; 1024-bit keys "are out there".

- **RSA is slow and bounded.** A modular exponentiation with a 2048-bit exponent costs thousands of big-number multiplications, and the message must be smaller than n, 256 bytes. So RSA encrypts a *symmetric key*, not a file: the **hybrid** scheme below.
- **Textbook RSA is deterministic.** The same m always gives the same c. A 4-digit PIN is recovered by encrypting all 10^4 candidates under the public key and comparing. Real RSA pads m with random bytes first (OAEP), which also blocks the algebraic tricks that come from $E(m_1)E(m_2) = E(m_1 m_2)$.
- **The primes must be chosen well.** A small prime is found by trial division; primes too close together fall to Fermat's method $n = a^2 - b^2$; $p = q$ makes n a square and φ wrong. The common public exponent is $e = 65537 = 2^{16} + 1$, prime and with only two 1-bits, so encryption is fast.



send ciphertext, encrypted key and IV; Bob recovers k with his private key, then m with AES

Anatomy: RSA, part by part

Part	What it does	If it is removed, weakened or misused
p, q large, random, distinct, similar size	n hard to factor, φ unknown	small n: trial division; p close to q: Fermat's $a^2 - b^2$; $p = q$: n a square, φ wrong; p or q shared with another key: gcd of two moduli factors both
$n = pq$ public	the modulus everyone computes in	nothing to hide; but leaking φ, d, or p breaks it instantly
$\varphi = (p-1)(q-1)$ secret	the exponent group's size, needed to compute d	using $n-1$ or $pq-1$: d is wrong and decryption fails; revealing φ: d and the factorisation follow
e coprime to φ	so that $d = e^{-1}$ exists	e even never works; $e = 1$ is no encryption; tiny e with tiny m and no padding: $m^e < n$, take an integer e-th root
$d = e^{-1} \bmod \varphi$ secret	undoes e by Euler	small d (to speed decryption): Wiener's attack recovers it from (n, e) when $d < n^{1/4}$
$m < n$	a residue is a message	$m \geq n$: ambiguous decryption; long messages must be split or, properly, handed to a hybrid scheme
Randomised padding (OAEP)	one m gives many c; kills the multiplicative structure	textbook RSA: dictionary attack on small message spaces; $E(m_1)E(m_2) = E(m_1 m_2)$ lets a chosen-ciphertext attacker unblind a target ciphertext
Hash before signing	fixed-size, non-malleable input to m^d	signing raw m: forge $\text{Sign}(m_1 m_2)$ from two signatures; existential forgery by picking s and publishing (s^e, s)
Separate key pairs for signing and encryption	no cross-protocol use	one pair for both: trick the holder into "decrypting" what is actually a signature on a chosen value

Go deeper

- e must be coprime to φ or d does not exist; that is why the algorithm checks $\gcd(e, \varphi) = 1$ before solving for d.
- The slide says "modulo a prime"; precisely, RSA works modulo the composite $n = pq$, and it is the compositeness that hides φ.
- Signing and encrypting use the same pair in opposite roles: encrypt with the public exponent, sign with the private one (Section 5).

Hard questions, answered

Q. Why combine AES and RSA for a long message?
A. RSA is slow and bounded by n; AES is fast and unbounded. RSA protects only the 32-byte key, AES does the bulk, and no pre-shared secret is needed.

Q. Eve knows (n, e) and the ciphertext of a 4-digit PIN. Attack?
A. Encrypt all 10^4 PINs and compare; textbook RSA is deterministic. Fix: randomised padding (OAEP).

4 Hash Functions and Passwords

4.1 What a hash is and the three things it must resist

A **cryptographic hash** H maps an input of any length to a fixed-length digest (SHA-256: 32 bytes). It is fast, lossy (the input cannot be reconstructed from the digest), and a one-bit change in the input changes about half the digest bits. Three resistance properties, from weakest to strongest:

- **Preimage resistance**: given y, hard to find any x with $H(x) = y$. Generic cost 2^n for an n-bit digest.
- **Second-preimage resistance**: given x, hard to find $x' \neq x$ with $H(x') = H(x)$. Generic cost 2^n.
- **Collision resistance**: hard to find *any* pair $m_1 \neq m_2$ with equal digests. Generic cost $2^{n/2}$.

Collisions exist, because the output space is finite and the input space is not; the question is only the work to find one. Collision resistance implies second-preimage resistance (a second preimage is a collision), and neither implies preimage resistance. MD5 (128 bits) and SHA-1 (160 bits) have practical collisions; SHA-2 (224/256/384/512) is current.

4.2 The birthday bound

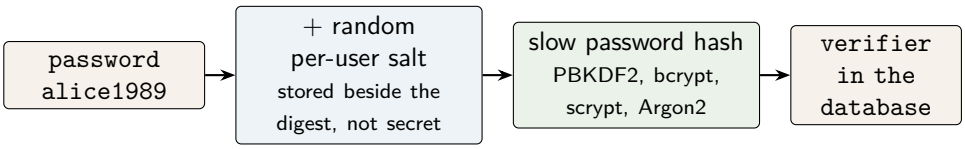
Why $2^{n/2}$ and not 2^n for collisions: among k random n-bit digests there are about $k^2/2$ pairs and each pair collides with probability 2^{-n}, so the expected number of collisions is $k^2/2^{n+1}$, which reaches 1 at $k \approx 2^{n/2}$. For MD5 that is 2^{64} work, within reach even if the function were perfect; preimages would still cost 2^{128}. This is why 128-bit digests are considered too short for signatures and 256-bit ones are the norm.

4.3 A hash alone authenticates nothing

A bare hash detects *accidental* corruption (a download checksum) and nothing adversarial. Integrity against an attacker needs a secret that Eve does not have, and Section 5 adds it.

Key insight. Everyone can compute H. If Alice sends $(m, H(m))$ and Eve controls the channel, Eve sends $(m', H(m'))$ and Bob's check passes.

4.4 Password storage



Store H(password), never the password; at login hash the input and compare. A stolen digest is harder to use than a plaintext, but preimage resistance assumes an unknown *high-entropy* input, and a password drawn from a small dictionary is found by enumeration however good H is. The **dictionary attack** precomputes the digests of common passwords; unsalted, the same password gives the same digest for every user, so one table serves every account and equal digests reveal shared passwords. A **salt** is a random per-account value stored next to the digest and mixed into the hash. It need not be secret; its job is to make each account's computation *different*, so there is no shared table and one guessing campaign per user. Salts do not strengthen a weak password; **slow** hashes do, by raising the cost of every guess (OpenSSL's `-pbkdf2` option is such a derivation, which is why the TP uses it).

Go deeper

- Hashing on the client before sending does not replace server-side hashing: the client-side hash *becomes* the password and can be replayed.
- "MD5 is broken" matters for signatures (an attacker crafts two documents with one digest; the 2008 rogue-CA certificate) more than for checksums against accidental corruption, where a second preimage would be needed.

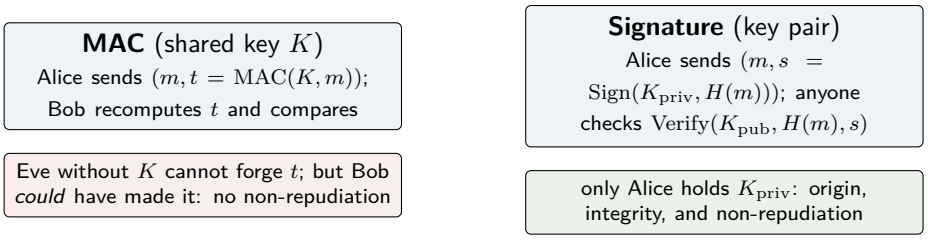
Hard questions, answered

Q. Two users share a password. Unsalted versus salted database?
A. Unsalted: identical digests, cracking one cracks both, precomputed tables apply to every account. Salted: different digests, separate work per account.

Q. Why is a 128-bit hash too short for collisions even if the function were perfect?
A. Birthday bound: 2^{64} work, feasible; preimages would still cost 2^{128}.

5 MACs and Digital Signatures

Integrity against an adversary needs a secret. There are two ways to add one, and they promise different things.



A **message authentication code** uses a shared secret. Constructions: the last block of CBC encryption of m (CBC-MAC), or hash-based schemes such as HMAC that mix a key into a cryptographic hash. A MAC gives integrity and authentication *between the two key holders*: Bob knows the message came from someone holding K, and since that is Alice or himself, from Alice. It gives no confidentiality, and no **non-repudiation**: both parties hold K, so Bob could have produced the tag, and a judge cannot tell who did.

A **digital signature** uses the sender's private key over the hash of the message and is verified with the sender's public key. Only Alice holds the private key, so a valid signature proves origin, integrity, and gives non-repudiation (NIST FIPS 186-4; algorithms RSA, DSA, ECDSA). No pre-shared secret is needed, only an authentic copy of Alice's public key.

5.1 Why sign the hash, not the message

Three reasons. Cost: a 15 MB document under 1024-bit RSA would need tens of thousands of slow operations, while the digest is 32 bytes. Size: the signature stays one RSA block whatever the message length. Safety: signing raw messages of the attacker's choice exposes RSA's multiplicative structure. The price is a dependence on collision resistance: if Eve can find m_1, m_2 with one digest and gets m_1 signed, the signature also validates m_2. This is exactly why MD5 signatures are dead.

	Confidentiality	Integrity	Authentication	Non-repudiation
Encryption	yes	no	no	no
MAC	no	yes	yes (shared key)	no
Signature	no	yes	yes	yes

Confidentiality and authenticity are independent: a secret sent anonymously has the first without the second; public information from a trusted source has the second without the first; secure communication needs both, which is why TLS encrypts *and* authenticates. None of the three gives availability.

Hard questions, answered

Q. A confidential contract that Bob cannot later deny having received from Alice. Primitives and order?
A. Sign with Alice's private key over $H(m)$ (origin, non-repudiation), then encrypt for Bob with the hybrid scheme. Sign first so the signature binds

the plaintext, not a ciphertext that could be re-wrapped.

Q. Is MAC $= H(K\|m)$ safe?
A. Not with Merkle–Damgård hashes (MD5, SHA-1, SHA-256): length extension lets Eve compute $H(K\|m\|\text{pad}\|m')$ without K. HMAC, or SHA-3, fixes it.

Q. Why can a MAC not serve as evidence in court?
A. The verifier holds the same key as the signer and could have produced the tag; a third party cannot attribute it. Only a signature attributes.

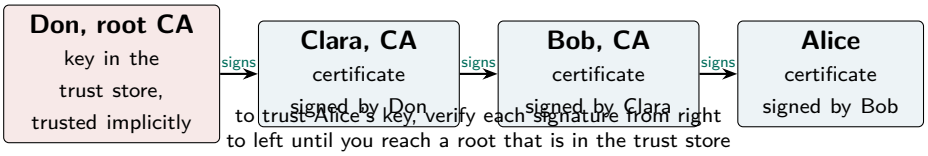
Anatomy: hashes, MACs and signatures, part by part

Part	What it does	If it is removed, weakened or misused
Fixed-length digest	compresses any input	too short (≤ 128 bits): birthday collisions in $2^{n/2}$
Preimage resistance	hides the input (passwords, commitments)	a hash that is a linear function or has a small output: invert it; low-entropy input (a PIN): enumerate it regardless
Collision resistance	one digest cannot stand for two documents	MD5/SHA-1: craft two documents with one digest, get the benign one signed
Avalanche	every output bit depends on every input bit	a hash that processes blocks independently (e.g. XOR of blocks): reorder or cancel blocks without changing the digest
Salt (public, per user)	no shared precomputed table	no salt: one table for all users, equal digests show shared passwords; one global salt: same problem per site
Slow hash (PBKDF2, bcrypt, Argon2)	raises cost per guess	fast hash (SHA-256): billions of guesses per second on a GPU
MAC key secret and shared	only key holders can produce tags	key leaks: forge anything; key = password reused elsewhere: same; $H(K\|m)$ with Merkle–Damgård: length extension
Signature with the *private* key over $H(m)$	origin, integrity, non-repudiation	signing with a shared key is a MAC, no non-repudiation; signing m not $H(m)$: forgeries from RSA's multiplicativity; verifying with an unauthenticated public key: Eve's signature verifies
Sign-then-encrypt ordering	signature binds the plaintext	encrypt-then-sign lets Eve strip Alice's signature and re-sign the ciphertext as hers

6 Certificates and PKI

6.1 The recursion

To verify Alice's signature, Bob needs her public key, and he needs to know it is *hers*. She cannot prove that remotely by signing something with the key in question: verifying would use that same key, which is what is in doubt. So someone else signs a statement binding Alice's identity to her key. That signer's key is vouched for by another signer, and so on, until a key that arrived by a different route and is trusted implicitly.



A **certificate** is a message containing an identity and a public key, signed by someone else. Each signer is a **certificate authority**; the implicitly trusted one is a **root CA**; the signatures form a **chain of trust**. The format is X.509 (RFC 5280). **PKI** (RFC 4949) is the whole apparatus, hardware, software, people, policies and procedures, needed to create, manage, store, distribute and *revoke* certificates. It includes the **trust store**: the list of root CAs and their keys, chosen by the operating-system or browser vendor and shipped with the software.

6.2 Getting a certificate

1. Generate a key pair; the private key never leaves the owner.
2. Send the public key and the identity to the CA in a **certificate signing request** (CSR), everything the certificate will contain except the signature, together with proof of identity.
3. The CA verifies the identity and signs.

If instead the CA generated the pair, it would hold the private key, and both the confidentiality and the non-repudiation guarantees of Section 5 would be void. The CSR exists so that no one but Alice ever holds Alice's private key.

6.3 Where the trust actually comes from

Whoever controls the trust store controls everything above it. An organisation adds a private root to sign its internal certificates. Malware that adds a root can have it sign a certificate for *any* site and mount a man-in-the-middle attack on every HTTPS connection from that machine. Some "security" appliances do exactly this on purpose to inspect encrypted traffic; the course calls it dangerous, because one compromised CA key then breaks confidentiality for everyone behind it. What a browser checks before trusting a server: the server certificate's signature by the intermediate, the intermediate's by a root in the

Key insight. The base case of the recursion is trust by distribution, not by cryptography: root keys arrive with the operating system.

store, validity dates, that the name matches the site, and revocation status; only then does it use the server's public key in the key exchange.

Hard questions, answered

Q. Why can't Alice self-sign to prove her key is hers?
A. Verifying would need her public key, the thing in question; a self-signature shows only that whoever holds the pair signed. Trust must enter from outside.

Q. Malware adds a root CA to one laptop. What can the attacker do, and what stays safe?
A. Issue a certificate for any site that laptop's browser will accept, hence intercept and alter every HTTPS connection from it. Other machines are unaffected; certificate pinning or fingerprint checks may detect it.

Q. What does revocation add that expiry does not?
A. Expiry handles the planned end of a certificate's life; revocation handles a private key compromised before that date. Without a revocation mechanism (CRL or OCSP) a stolen key stays valid until expiry.

Anatomy: certificates and PKI, part by part

Part	What it does	If it is removed, weakened or misused
CA signature over (name, key)	binds the identity to the key	self-signed leaf: proves nothing about the name; a CA that does not verify identity: anyone gets a certificate for any name
Chain to a root	reduces every check to one trusted key	a missing intermediate: verification fails; an intermediate allowed to sign any name: one compromise signs the world
Trust store	the implicitly trusted roots	an added rogue root: MITM on every HTTPS site from that machine; a root key leak: same, everywhere
CSR (key generated by the owner)	private key never leaves the owner	CA-generated keys: the CA can decrypt and sign as the owner
Validity dates and revocation	limits the damage window of a stolen key	no revocation checking: a stolen key stays trusted until expiry
Name check	the certificate must name the site you asked for	a valid certificate for `evil.com` presented by `bank.com`: accepted if the client skips the name check

7 Steganography

Cryptography hides the *meaning* of a message; steganography hides its *existence*. Alice and Bob communicate through a warden; the innocent carriers are **stego objects**, the hidden data the **payload**. A *passive* warden only

History. Simmons (1983) framed it as the Prisoners' Problem; the first recorded technique is Greek, around 440 BC.

observes, which makes it a detection problem for the warden; an *active* warden alters the objects, which makes it a robustness problem for Alice. Null ciphers hide a message in the n-th letters of words: the third letters of "Fishing freshwater bends and saltwater coasts rewards anyone feeling stressed…" spell "Send Lawyers, Guns, and Money", and the first letters of one paragraph and second letters of the next in a WWI telegram spell "PERSHING SAILS FROM N.Y. JUNE 1". Carriers today: text, image, audio, video, network protocols, email; the size of the Internet is what makes hiding effective.

LSB steganography overwrites the least significant bit of each pixel or audio sample with one payload bit. Each value changes by at most 1 in 255, invisible to the eye; capacity is one bit per byte; it is detectable by LSB statistics and destroyed by any recompression. Encrypt the payload first, so that detection does not reveal meaning.

Worked example: Hiding one letter

'A' is `01000001`. Pixels `10110010, 10110011, 10110010, 10110011, 10110010, 10110011, 10110010, 10110011`. Replace each LSB with the next payload bit: `10110010, 10110011, 10110010, 10110010, 10110010, 10110010, 10110010, 10110011`. Four pixels changed, each by one unit.

8 TP Companion: OpenSSL Missions 1 and 3

Technique: Mission 3, command by command

```
openssl genrsa -out mySmall.pem 512          # 5.1: private key,
    UNENCRYPTED on disk
openssl rsa -in mySmall.pem -text -noout     # 5.2: prints n, e,
    d, p, q and the CRT values
openssl rsa -pubin -in otherSmall_pub.pem -noout -text   # 3.4: n
     and e of the 50-bit key
openssl genrsa -out private1.pem 2048                    # 5.6:
    plain
openssl genrsa -aes256 -out private2.pem 2048            # 5.6:
    file encrypted with AES-256-CBC
openssl rsa -in private1.pem -pubout -out public1.pem    # 5.8:
    extract the public key
openssl pkeyutl -encrypt -pubin -inkey public1.pem -in msg.txt -
    out msg.enc   # 5.11
openssl pkeyutl -decrypt -inkey private1.pem -in msg.enc -out msg
    .dec
```

- **5.3** The PEM file holds the private key in clear; anyone who reads the file owns the key. `-aes256` at generation wraps it under a passphrase (Section 3's hybrid idea applied to a key file).
- **3.4** Factor the 50-bit modulus: trial-divide up to $\sqrt{n} \approx 3.4 \times 10^7$, a second of Python; then $\varphi = (p-1)(q-1)$ and $d = e^{-1} \bmod \varphi$ with `pow(e, -1, phi)`. **5.4** The same loop on a 512-bit modulus runs for

2^{256} steps: never.

- **5.9** A public key is public by design; storing it in plaintext is fine. What must be protected is its *authenticity*, which is what certificates do.
- **5.12** `pkeyutl` refuses a file larger than the modulus minus padding (about 245 bytes for 2048 bits): RSA encrypts one block and is slow.
- **5.15** Hybrid: `openssl rand -hex 32` for the AES key and `-hex 16` for the IV; `openssl enc -aes-256-cbc -K <key> -iv <iv> -in long.txt -out long.enc`; encrypt the key file with the partner's public key via `pkeyutl`; send ciphertext, encrypted key, IV. The partner decrypts the key with their private key, then the message with `enc -d`.

Mission 1 is the worked example of Section 3; Mission 2 (ECB on the image) is in Chapter 4.

9 Tricks, Techniques and Classic Problems

Warning / Gotcha: Common student mistakes

- **Computing d modulo n instead of modulo φ.** $ed \equiv 1 \pmod{\varphi(n)}$, never $\pmod{n}$.
- **Using $\varphi = pq - 1$.** It is $(p-1)(q-1)$.
- **Raising to the power as an integer.** Reduce after every squaring; 26^7 never has to exist.
- **"Encrypted, therefore authentic."** Encryption gives confidentiality only; a MAC or signature gives integrity and origin.
- **"Hashed, therefore tamper-proof."** Anyone can recompute a keyless hash; it detects accidents, not attackers.
- **Calling a MAC non-repudiable.** Both parties hold the key.
- **Treating the salt as a secret, or as a strengthener of weak passwords.** It is public and only prevents shared precomputation; slow hashing is what raises the per-guess cost.
- **Thinking a certificate proves honesty.** It binds a name to a key, signed by a CA you chose to trust; it says nothing about the key holder's intentions.

Worked example: Classic exam problems

P1. $p = 11$, $q = 13$, $e = 7$. Keys, and encrypt $m = 9$.
$n = 143$, $\varphi = 120$, $d = 103$ ($7 \cdot 103 = 721 = 6 \cdot 120 + 1$). $9^7 \bmod 143$: $9^2 = 81$, $9^4 = 6561 \equiv 6561 - 45 \cdot 143 = 126$, $9^7 \equiv 126 \cdot 81 \cdot 9 = 91{,}854 \equiv 48$. So $c = 48$.

P2. Why is $e = 2$ never allowed?
$\varphi = (p-1)(q-1)$ is even, so $\gcd(2, \varphi) = 2$ and d does not exist.

P3. An attacker learns $\varphi(n)$ for a 2048-bit RSA key. Is the key broken?
Yes: $d = e^{-1} \bmod \varphi$ is immediate, and p, q follow from $p + q = n - \varphi + 1$

and $pq = n$ (a quadratic).

P4. 200 users, how many symmetric keys versus key pairs, and what replaces the secure channel?

$200 \cdot 199/2 = 19{,}900$ versus 200 pairs; an authentic channel (certificates) replaces the secret one.

P5. A download page shows a file and its SHA-256. An attacker controls the page. Is the checksum useful?

No: the attacker replaces both file and digest. Useful only if the digest comes over an independent authentic channel or the file is signed.

P6. Alice signs a document with RSA-SHA-256; Bob verifies; later Alice claims she never signed. Who is right, and under what assumption?

Bob, assuming Alice's private key was never compromised and her public key is certified; non-repudiation rests on sole possession of the private key.

P7. 64-bit digest. Work for a preimage, and for a collision?

2^{64} and 2^{32} (birthday); the second is seconds on a laptop.

P8. A corporate proxy decrypts all HTTPS. What had to be installed on every laptop and what is the risk?

The proxy's root certificate in each trust store; the proxy's private key now unlocks every employee's traffic, and if it leaks, so does everything.

Summary: Chapter 5 in six lines

1. A key pair replaces $N(N-1)/2$ shared secrets by $2N$ keys and trades a secret channel for an authentic one.
2. RSA: $n = pq$, $\varphi = (p-1)(q-1)$, $ed \equiv 1 \pmod{\varphi}$; correct by Euler; secure by factoring; used hybrid with AES and with padding.
3. A hash has preimage, second-preimage and collision resistance (2^n, 2^n, $2^{n/2}$); it is keyless, so it authenticates nothing; passwords get a public salt and a slow hash.
4. A MAC (shared key) gives integrity and authentication between the key holders; a signature (private key over $H(m)$) adds non-repudiation; encryption gives confidentiality only.
5. Certificates bind a name to a key by a CA's signature; chains end at a root that arrived with the OS; PKI includes revocation; the trust store is the root of everything.
6. Steganography hides existence rather than meaning; LSB changes each byte by at most one; encrypt the payload.